NEXUS · DOCUMENTATION PACKAGE · DELIVERABLE 11

Deployment Guide

How the API and app are actually deployed today (local), the container path the repository supports, and labelled recommendations for public hosting.

Document control

Document ID	NX-11-DEP
Version	2.0
Date	2026-11-05
Status	Draft for team review (not approved)
Intended readers	Developers and operators setting up the API and building the app
Evidence baseline	Branch New-Changes: commit 716bedb plus the uncommitted Part 3 working-tree changes of 2026-11-05 (v1.0 analysed 3a3f275)
Prepared by	AI-assisted (Claude Code) from repository evidence; team review pending
Approved by	Unknown No approval recorded

Evidence labels used in this document: Implemented verified in code, configuration or test output; Inferred reasoned from evidence but not executed; Recommendation proposed, not implemented; Open question needs a decision; Unknown no evidence available; Estimate planning figure, not a measurement.

Contents

1. 1 Current deployment position
2. 2 Prerequisites
3. 3 Run the API locally (DEP-01)
    1. 3.1 Point the app at the local API
4. 4 Run the API in a container (DEP-02)
5. 5 Public hosting (recommendations)
    1. 5.1 Building a release that uses the hosted API
6. 6 Rollback
7. 7 Enabling Google SSO, push and signing (v2.0)

1 Current deployment position

> **No hosting platform is configured.** The repository contains no provider-specific configuration: no cloud manifests, infrastructure-as-code or deploy workflow. The README states that public deployment "still needs a hosted API URL" and that the earlier production hostname "was not a configured service" Implemented. Sections 3 and 4 document what the repository supports, and section 5 contains recommendations only.

Supported deployment targets

ID	Target	Defined by	Verified
DEP-01	Local Node process (development)	`backend/package.json` `start` / `dev`, `docs/ONLINE_OFFLINE.md`	Server module loaded and tested in process (16/16) Implemented; listening server not started for this package
DEP-02	Single container with a named volume	`backend/Dockerfile`, `backend/compose.yaml`	Not executed Unknown; CI smoke test never succeeded (CI-02)
DEP-03	Root `docker-compose.yml`	Repository root	Fixed in v2.0 to match DEP-02 (named volume at `/data`); config validated, not run (AUD-02)
DEP-04	Android debug/release builds	`app/build.gradle.kts`, Android CI	Debug build last green in CI on 2026-09-24; release never built in the retrieved history

2 Prerequisites

- API: Node.js 22 or later (required by google-auth-library 11 and firebase-admin 14); npm. For containers: Docker with Compose v2.
- App: Android Studio with SDK Platform 37; JDK 17+ (the build uses a 17 toolchain); an emulator or device on API 24+.
- A strong `JWT_SECRET` of 32+ characters, stored outside git (`.env*` files are git-ignored).

3 Run the API locally (DEP-01)

```
# PowerShell, from backend/
npm.cmd ci
$env:JWT_SECRET = node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
npm.cmd start          # listens on http://localhost:5263
curl.exe http://localhost:5263/health
```

- Data is written to `backend/nexus-data.json` unless `DATABASE_PATH` is set. The file is git-ignored.
- Keep the same secret across restarts, or every session becomes invalid (RB-06).
- Optional test accounts: run `docs/seed_test_users.ps1` against a **local** API only (AUD-14).

3.1 Point the app at the local API

Debug build targets

Device	Command

Device	Command
Emulator	Run from Android Studio, or `.\gradlew.bat :app:assembleDebug`. The default URL `http://10.0.2.2:5263/api/` reaches the host.
Phone on the same LAN	`.\gradlew.bat :app:assembleDebug -PNEXUS_API_URL=http://<your-LAN-IP>:5263/api/`, and allow port 5263 through the firewall.

The URL must end in `/` and match `https?://host[:port]/path/`, or the Gradle build fails at configuration time (TR-07).

4 Run the API in a container (DEP-02)

```
# from backend/, with JWT_SECRET exported in the shell or a git-ignored .env file
docker compose up --build -d
docker compose ps          # STATUS should become "healthy"
curl http://localhost:5263/health
```

- The image is `node:22-alpine`. It installs production dependencies only, runs as the non-root `node` user, exposes 5263, and stores data at `/data/nexus-data.json` on the `nexus-data` named volume.
- A Docker `HEALTHCHECK` calls `/health` every 30 s. `restart: unless-stopped` is set.
- Compose refuses to start if `JWT_SECRET` is unset (`${JWT_SECRET:?…}`).
- Run **one** replica only (CON-01). Back up the volume (RB-04).

5 Public hosting (recommendations)

Everything in this section is **Recommendation**. It describes the minimum the code requires from any host; it does not select a provider.

Host requirements derived from the code

ID	Requirement	Why
DEP-R1	Runs a Node 20+ process or an OCI container, with exactly one instance	In-memory state and single-file store (CON-01)
DEP-R2	Persistent disk or volume mounted at `/data` (or `DATABASE_PATH`) that survives redeploys	Ephemeral file systems would lose all data
DEP-R3	HTTPS termination with a valid certificate and a stable domain	Release builds reject HTTP (NFR-02)
DEP-R4	Secret store for `JWT_SECRET`; `NODE_ENV=production`	Startup check; production error masking
DEP-R5	Health check against `/health`	Liveness
DEP-R6	Configure Express `trust proxy` before go-live	Rate limiting behind the proxy (AUD-06, IMP-06)
DEP-R7	Fix AUD-01 and set up scheduled backups before real users	Data-loss risk

5.1 Building a release that uses the hosted API

1. Confirm `https://<host>/health` responds.

2. Locally: `.\gradlew.bat :app:assembleRelease -PNEXUS_API_URL=https://<host>/api/` ; or set the GitHub repository variable `NEXUS_API_URL` so CI builds the release APK and AAB (CI-01).
3. Sign the artifacts. Signing is not configured in the repository (AUD-08).
4. Complete the release checklist (REL-CHECK).

6 Rollback

No versioned deployments exist, so rollback is manual **Implemented**. **Recommendation** Tag each deployed commit. To roll back, redeploy the previous tag against the same data volume. Data written by a newer version stays in the file (the JSON store has no schema version), so take a backup before every deploy.

7 Enabling Google SSO, push and signing (v2.0)

One-off setup per environment (Recommendation: the accounts are the team's to create)

Feature	Steps
Google SSO	1. Google Cloud Console → APIs & Services → Credentials: create an OAuth client of type **Web application** (its ID is used by both app and API) and one of type **Android** for package `com.example.nexus` with the SHA-1 of the debug key and of the upload key (printed by `scripts/create-upload-keystore.ps1`). 2. API: `GOOGLE_CLIENT_ID=<web-id>` . 3. App: `-PGOOGLE_WEB_CLIENT_ID=<web-id>` . 4. Check: `GET /api/app/config` shows `googleSignIn: true` .
Push (FCM)	1. Firebase Console: add an Android app `com.example.nexus` , download `google-services.json` into `app/` (git-ignored). 2. Project settings → Service accounts → generate key; set `FIREBASE_SERVICE_ACCOUNT_BASE64` to the base64 of that JSON on the API host. 3. Check: `pushNotifications: true` in `/api/app/config` , then Settings → Send test notification reports "Push sent to n device(s)".
Release signing	Run `scripts/create-upload-keystore.ps1` once (done on the development PC on 2026-11-05); back up `%USERPROFILE%\.nexus\nexus-upload.jks` and `keystore.properties` . In CI, provide `NEXUS_KEYSTORE_PATH` , `NEXUS_KEYSTORE_PASSWORD` , `NEXUS_KEY_ALIAS` and `NEXUS_KEY_PASSWORD` from secrets.
Proxy	Behind an HTTPS reverse proxy set `TRUST_PROXY=1` (DEP-R6).